

From passive *documentation* to AI-native *reasoning* systems.

An engine that reasons over structured knowledge, validates against truth, and returns deterministic, non-hallucinated answers — even for messy multilingual input.

Author

Puneet Khamborkar

Focus

Applied AI • Systems • Product

// Core stack

llm → reasoning layer

kdf → structured truth

val → grounding engine

Docs don't think. *Naive LLMs* hallucinate.

The gap between static documentation and generative AI is where enterprise trust breaks.

01 • PASSIVE DOCS

Search, don't answer.

Knowledge is buried in PDFs, wikis, and stale pages. Users keyword-match and guess — no intent, no reasoning, no closure.

02 • NAIVE LLMs

Fluent, but unreliable.

Plain chatbots fabricate fixes, confuse versions, and drift across sessions. Beautiful prose, broken truth — untrustworthy in production.

03 • THE GAP

No grounding layer.

Between user intent and factual answer, there's no validation, no schema, no determinism. Enterprises cannot ship what they cannot verify.

Don't replace the LLM. *Constrain* it with truth.

The shift is architectural — three layers, one contract: reasoning speaks, knowledge governs, validation decides.

// The equation

LLM · reasoning + **KDF** · truth + **Validator** · safety

LAYER 01 · REASONING

LLM interprets intent

Parses noisy, multilingual queries into structured intent.

LAYER 02 · TRUTH

KDFs hold knowledge

Machine-readable definitions of causes, fixes, and outcomes.

LAYER 03 · SAFETY

Validator grounds output

Every answer reconciled to a KDF — or rejected.

Six stages. One *safe* answer.

Every query flows through a deterministic pipeline — reasoning is bounded by retrieval, retrieval is verified by validation.



// INTELLIGENCE

LLM reasons freely — but only within the space of intents the system knows how to serve.

// GOVERNANCE

KDFs are versioned, reviewable source-of-truth — curated by humans, consumed by machines.

// SAFETY

No KDF match → no answer. The system prefers silence over hallucination.

Knowledge, *as code.*

A KDF encodes an **intent**, its **failure modes**, probable **causes**, and verified **resolution steps** — reviewable by humans, executable by machines.

FIELD

intent

what the user means

FIELD

failure

what broke

FIELD

causes

why it broke

FIELD

resolution

how to fix

- **Deterministic.** Same input class → same structured output.
- **Reviewable.** Git-friendly YAML — subject-matter experts own truth.
- **Composable.** KDFs scale independently of the underlying LLM.



kdf/task_not_saving.yaml

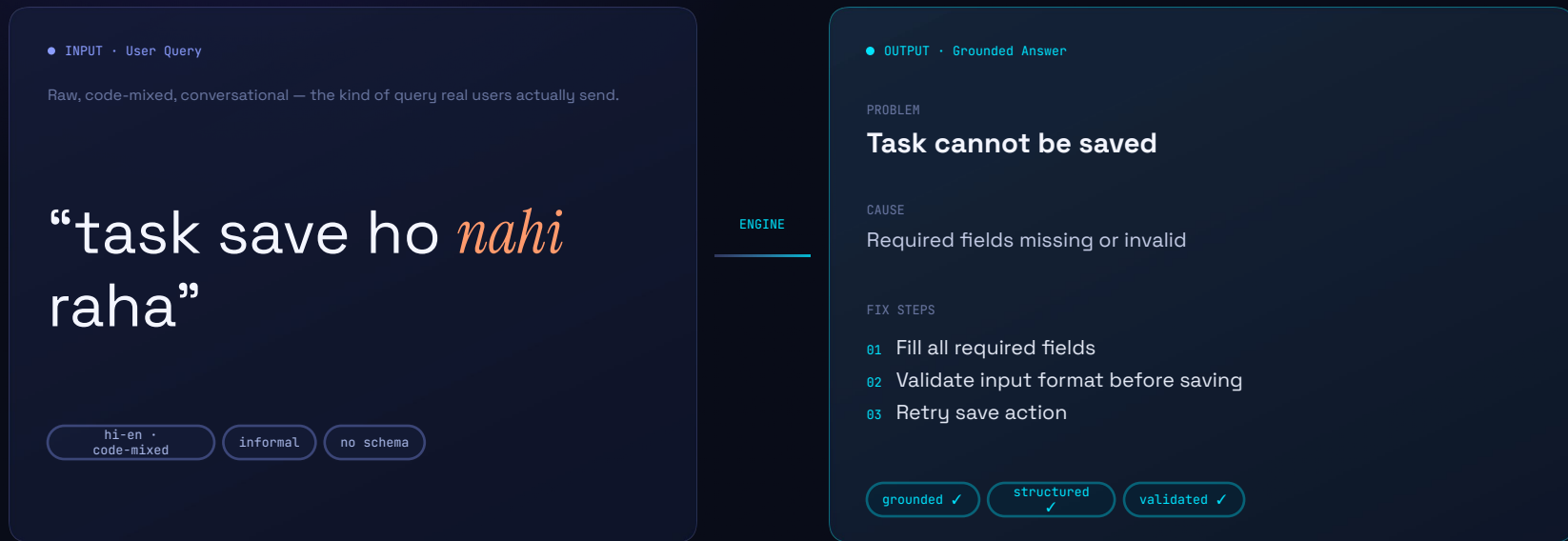
```
intent:
  name: task_not_saving

failure:
  problem: Task cannot be saved
  causes:
    id: validation_error
    label: Required fields missing

resolution:
  validation_error:
    steps:
      - Fill all required fields
      - Validate input format
      - Retry save action
```

Messy input. *Precise* output.

A real Hindi-English code-mixed query, routed through the engine — end to end, zero hallucination.



Built to ship.

Runs locally in minutes.



ai-native-knowledge-engine/

AcmeTasker_MVP/

```
├── acmetasker-ui/ # Next.js frontend
├── backend/      # AI engine + APIs
│   ├── app.py   # Flask server
│   └── kdf/     # Truth layer (YAML)
│       └── task_not_saving.yaml
├── requirements.txt
└── README.md
```

// STACK

Next.js

Python · Flask

LLM Reasoning

Ollama · Local LLM

YAML · KDF

REST API

Validation Engine

Whisper-ready

// RUN IT LOCALLY

```
$cd backend && pip install -r ../requirements.txt
```

```
$python3 app.py # :5001
```

```
$cd acmetasker-ui && npm install && npm run dev
```

```
$curl -X POST :5001/ask-d '{"query":"..."}'
```

From AI assistants to *decision engines*.

The same four properties every enterprise AI team is asked to prove — baked into the architecture, not bolted on.

PROPERTY • 01

0x

Hallucinations

Unmatched intents return nothing, not fiction.

PROPERTY • 02

1:1

Determinism

Same intent class → same structured output, every time.

PROPERTY • 03

∞

Auditability

Every answer traces back to a versioned KDF in git.

PROPERTY • 04

Local

Sovereignty

Runs on Ollama — no data leaves the enterprise boundary.

Let's build *smarter* systems. Together.

// ROADMAP

Whisper · voice
input

Image-based
debugging

Auto-learning
KDFs

Enterprise
integrations

Observability + feedback
loops

AUTHOR

Puneet Khamborkar

Building AI-native systems at the intersection of technical writing, product thinking, and applied AI.

// LET'S CONNECT

**Open to discuss
production AI systems
& reliability.**

- Connect on [LinkedIn](#)
- Star the [GitHub repo](#)
- DM for [collaboration](#)